Skills in this repository

69 Claude Code skills live in `.claude/skills/`. Claude loads one automatically when your request matches what it is for, or you can call one by name (`/caveman`, `/session-handoff`, ...). Each was copied from an open-source collection, kept with its license file, and reviewed before adding (see Sources and licenses at the end).

Quick picks

I want to...	Say something like	Skill
Make the session last longer	"be brief" / `/caveman`	`caveman`
Continue tomorrow in a fresh session	"write a handoff"	`session-handoff`
Shrink CLAUDE.md so every session starts lighter	`/caveman-compress CLAUDE.md`	`caveman-compress`
Look at a new dataset	"explore this CSV"	`exploratory-data-analysis`
Test a hypothesis properly	"is the difference significant?"	`statistical-analysis`
Build and explain a model	"train a classifier", "why did it predict this?"	`scikit-learn`, `shap`
Forecast a time series	"forecast the next 12 months"	`timesfm-forecasting`, `statsmodels`
Simulate a queue or supply chain	"simulate this warehouse"	`simpy`
Find a process bottleneck	"where does work wait?"	`process-mapper`
Check why the website does not rank	"SEO audit of index.html"	`seo-audit`
Fix a bug without guessing	"this test fails"	`systematic-debugging`
Keep code changes small and replies short	"ponytail", "simplest solution"	`ponytail`
Make a page look polished	"critique and polish this page"	`impeccable`
Check a site for accessibility	"a11y audit"	`accessibility`
Look for security holes	"security review of this repo"	`security-audit`

1. Save tokens and make sessions last longer (7)

A session ends when its context window fills up. These skills cut what goes into it, or let you continue cleanly in a new session.

Skill	What it does	How to use it
`caveman`	Short, direct answers: no greetings, recaps or filler; code, numbers and errors kept exact. Stays on until you say "stop caveman" or "normal mode". It switches back to full sentences for security warnings and irreversible actions.	`/caveman`, "be brief", "less tokens"

Skill	What it does	How to use it
`caveman-compress`	Rewrites a memory file (CLAUDE.md, a todo list, notes) in compressed form so it costs fewer tokens every time it is loaded; keeps a readable backup outside the repo. It sends the file's text to Claude to compress it and refuses files that look like secrets.	`/caveman-compress <path>`
`caveman-review`	Code review in one line per finding: location, problem, fix.	`/caveman-review`, "review the diff"
`session-handoff`	Writes a handoff note (goal, current state, decisions, next steps, skills to load), pointing to files instead of copying them and redacting secrets, so a fresh session picks up where this one stopped.	"write a handoff", "I'm running low on context"
`dispatching-parallel-agents`	Splits independent tasks across sub-agents; their work happens in their own context, and only the results come back.	"do these three things in parallel"
`subagent-driven-development`	Runs each task of a plan in a fresh sub-agent with a review between tasks, keeping the main conversation small.	after `writing-plans`: "execute the plan with subagents"
`ponytail`	Coding in "lazy senior dev" mode: the smallest change that fully solves the task and a reply you can read in one pass. It applies to coding tasks by default (levels lite, full, ultra).	"ponytail", "be lazy", "simplest solution"

Habits that save more than any skill

- One task per session; start a new session (with a handoff) when the topic changes.
- Point Claude to files and line numbers instead of pasting their contents.
- Ask for summaries of long outputs (logs, test runs) rather than the full text.
- Keep CLAUDE.md short; compress it with `caveman-compress` once it grows.
- Use `/compact` before the context is full, not after.

2. Data analysis and statistics (6)

Skill	Use it for
`exploratory-data-analysis`	First look at a data file: profile, missing values, leakage and outlier audits, an EDA report outline
`statistical-analysis`	Choosing the right test, checking assumptions, effect sizes, writing up results
`statistical-power`	How many samples you need; minimum detectable effect; power curves
`experimental-design`	Planning an experiment or A/B test before collecting data: randomization, blocking, factorial designs
`statsmodels`	Regression, GLMs, ARIMA/SARIMAX with proper inference and diagnostics
`pymc`	Bayesian models and uncertainty

3. Machine learning and forecasting (5)

Skill	Use it for
`scikit-learn`	Classification, regression, clustering, pipelines, cross-validation, tuning
`shap`	Explaining a model's predictions, globally and for single cases
`scikit-survival`	Time-to-event models: churn, failures, cancellations
`timesfm-forecasting`	Zero-shot forecasts with Google's TimesFM, with quantiles and a held-out check (downloads the model)
`umap-learn`	2D/3D maps of high-dimensional data; preprocessing for clustering

4. Data handling and big data (5)

Skill	Use it for
`polars`	Fast DataFrame work and pandas migration
`dask`	Data larger than memory; parallel processing
`geopandas`	Maps, spatial joins, provinces and coordinates
`networkx`	Graphs: franchise links, routes, networks, centrality, communities
`markitdown`	Converting PDF, Word, Excel or PowerPoint files to Markdown for analysis

5. Charts, diagrams and writing (6)

Skill	Use it for
`matplotlib`	Fine-grained static charts
`seaborn`	Quick statistical charts
`scientific-visualization`	Figure design: honest axes, colorblind-safe palettes, multi-panel layouts
`markdown-mermaid-writing`	Markdown documents and Mermaid diagrams (flowcharts, timelines, schemas)
`scientific-writing`	Drafting and checking reports or papers with traceable evidence
`literature-review`	Systematic literature searches, screening and citation checks (uses PubMed, arXiv, Crossref; its optional diagram script needs an OpenRouter key)

6. Operations, simulation and optimization (5)

Skill	Use it for
`simpy`	Discrete-event simulation: queues, resources, warm-up, replications
`pymoo`	Multi-objective optimization (cost vs service, Pareto fronts)
`process-mapper`	Mapping a business process, measuring cycle times, finding bottlenecks
`capacity-planner`	Staffing and capacity with Erlang C queueing; utilization risk; hiring order
`procurement-optimizer`	Spend review, supplier consolidation, slow purchasing cycles

7. Business and research reports (2)

Skill	Use it for
`market-research-reports`	Market sizing (TAM/SAM/SOM), competitive landscape, evidence-traceable reports
`vendor-management`	Vendor scorecards, SLA compliance, third-party risk tiers

8. Website: SEO, analytics and testing (4)

Skill	Use it for
`seo-audit`	Why a page does not rank: technical and on-page checks (`seo_checker.py --file page.html`)
`programmatic-seo`	Many pages from one template and data, without thin-content penalties
`analytics-tracking`	GA4 / Google Tag Manager events and conversion tracking plans
`webapp-testing`	Testing pages in a real browser with Playwright (screenshots, console logs)

9. Engineering practice (7)

Skill	Use it for
`systematic-debugging`	Finding the root cause before proposing a fix
`test-driven-development`	Writing the failing test first
`verification-before-completion`	Running the checks before saying something works
`writing-plans`	Turning a spec into a step-by-step plan before coding
`executing-plans`	Carrying out a written plan step by step yourself
`requesting-code-review`	Getting a review before merging
`receiving-code-review`	Weighing review comments instead of applying them blindly

10. Making new skills (1)

Skill	Use it for
`skill-creator`	Writing, testing and improving a new skill for this repo

11. From the Collective Brain catalogue (21 more, plus `ponytail` above)

Picked from collectivebrain.de/en/skills and copied from each author's own repository. The full catalogue of all 95 skills listed there, with the status of each, is in `docs/collectivebrain_skills.md` .

Web design and quality

Skill	Use it for
`impeccable`	Designing, critiquing, auditing and polishing a frontend or design system. It runs a small analysis engine that it downloads from GitHub, pinned to one version and checked against SHA-256 hashes shipped with the skill

Skill	Use it for
`accessibility`	WCAG 2.2 audit of a website and fixes in the source
`emil-design-eng`	UI polish, component design and animation decisions (Emil Kowalski's approach)
`apple-design`	Apple-style gestures, spring animations, materials and typography on the web
`review-animations`	Strict review of animation code; runs only when you call it by name
`security-audit`	Security review of a codebase, API or service (Cloudflare)

Code and tools (most of these only matter in projects that use that tool)

Skill	Use it for	Needs
`tdd`	Test-first development, red-green-refactor (Matt Pocock)	
`setup-matt-pocock-skills`	One-time setup of issue tracker and labels for Matt Pocock's engineering skills; runs only when called by name	
`typescript-advanced-types`	Generics, conditional and mapped types in TypeScript	
`playwright-cli`	Browser automation and Playwright tests from the command line	`@playwright/cli`
`shadcn`	shadcn/ui components and design systems	a shadcn project
`turborepo`	Turborepo monorepos: pipelines, caching	Turborepo
`supabase-postgres-best-practices`	Postgres schemas, migrations, RLS, indexes and slow queries	Postgres
`obsidian-markdown`	Obsidian notes: wikilinks, callouts, properties	Obsidian
`obsidian-bases`	Obsidian Bases (`.base` files): views, filters, formulas	Obsidian
`google-agents-cli-adk-code`	Writing agents with Google's Agent Development Kit	`agents-cli`
`google-agents-cli-deploy`	Deploying ADK agents (Agent Runtime, Cloud Run, GKE)	`agents-cli`, Google Cloud
`google-agents-cli-publish`	Registering agents with Gemini and Agent Registry	`agents-cli`
`azure-ai`	Azure AI Search, Speech, OpenAI and Document Intelligence apps	Azure
`lark-im`	Lark/Feishu messages and group chats (description in Chinese)	Lark CLI
`herdr`	Controlling the Herdr terminal multiplexer; only when you mention Herdr	Herdr

Not copied from that catalogue, and why: 56 skills are hosted on collectivebrain.de with no license to republish them, so they are not stored here; install the ones you want on your own machine with `scripts/install_collectivebrain_skills.sh`. `find-skills` installs unvetted skills with `npx` on any "how do

I" question; `agent-browser` tells Claude to prefer it over built-in web tools but needs a CLI that is not installed; `task-observer` runs during every multi-step task and writes logs; the SEO/GEO/AEO audit has no license.

Sources and licenses

Collection	Skills	License	Repository	Commit copied
K-Dense scientific skills	25	MIT	https://github.com/K-Dense-AI/claude-scientific-skills	`2d2065f`
obra/superpowers	9	MIT	https://github.com/obra/superpowers	`22f0b46`
alirezarezvani/claude-skills	8	MIT	https://github.com/alirezarezvani/claude-skills	`08cd8a9`
caveman (Julius Brussee)	3	Apache 2.0	https://github.com/JuliusBrussee/caveman	`ad47375`
Anthropic skills	2	Apache 2.0	https://github.com/anthropics/skills	`2c5958e`
emilkowalski/skills	3	MIT	https://github.com/emilkowalski/skills	`f7ce8c0`
google/agents-cli	3	Apache 2.0	https://github.com/google/agents-cli	`6eedbdc`
mattpocock/skills	2	MIT	https://github.com/mattpocock/skills	`5863806`
kepano/obsidian-skills	2	MIT	https://github.com/kepano/obsidian-skills	`3981277`
pbakaus/impeccable	1	Apache 2.0	https://github.com/pbakaus/impeccable	`c4cd192`
addyosmani/web-quality-skills	1	MIT	https://github.com/addyosmani/web-quality-skills	`afa8da9`
cloudflare/security-audit-skill	1	MIT	https://github.com/cloudflare/security-audit-skill	`c1c8a8c`
dietrichgebert/ponytail	1	MIT	https://github.com/dietrichgebert/ponytail	`96608e0`
wshobson/agents	1	MIT	https://github.com/wshobson/agents	`9c61ab2`
microsoft/playwright-cli	1	Apache 2.0	https://github.com/microsoft/playwright-cli	`b217248`
microsoft/azure-skills	1	MIT	https://github.com/microsoft/azure-skills	`40a30c5`
shadcn/ui	1	MIT	https://github.com/shadcn/ui	`e62c943`
vercel/turborepo	1	MIT	https://github.com/vercel/turborepo	`805ec38`
supabase/agent-skills	1	MIT	https://github.com/supabase/agent-skills	`c9be0e9`
larksuite/cli	1	MIT	https://github.com/larksuite/cli	`3d82c85`
herdrdev/herdr	1	Apache 2.0	https://github.com/herdrdev/herdr	`ee5bfb5`

Each skill folder keeps its license file (the caveman and impeccable skills also keep their project's `NOTICE`).

Changes from upstream: in the obra/superpowers skills, references written as `superpowers:<skill>` now use the plain skill name, and references to superpowers skills not copied here (`finishing-a-development-branch`, `using-git-worktrees`, `writing-skills`) are marked as not installed. Nothing else was changed. The `caveman` skill mentions `ultracave` and `megacave` modes; those skills were not copied, so use plain `caveman`.

Reviewed and left out: Anthropic's `docx`, `pdf`, `pptx` and `xlsx` (source-available, not open source); superpowers' `brainstorming` (it makes itself run before any creative task); caveman's Caveman Cloud skills (need its hosted service); `token-optimizer` (PolyForm Noncommercial license, not open source).

What the scripts do: they run locally. The ones that reach the network do so only for their stated job: `seo-audit` fetches a page when you give it a URL, `literature-review` queries literature databases, `timesfm-forecasting` downloads the TimesFM model, `caveman-compress` and `skill-creator` call Claude.

To update a skill, copy its folder again from the repository above and update the commit here.